



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
"МИРЭА - Российский технологический университет"

РТУ МИРЭА

Институт информационных технологий (ИТ)
Кафедра инструментального и прикладного программного обеспечения

**ОТЧЕТ
ПО ПРАКТИЧЕСКОЙ РАБОТЕ №4
по дисциплине
«Технологии виртуализации клиент-серверных приложений»**

Выполнил студент группы ИКБО-16-22

Грибков А.С.

Принял преподаватель кафедры ИиППО

Волков М.Ю.

Практические работы выполнены

«__»_____2025 г.

«Зачтено»

«__»_____2025 г.

СОДЕРЖАНИЕ

1 ЦЕЛЬ РАБОТЫ.....	4
2 КРАТКАЯ ТЕОРЕТИЧЕСКАЯ ЧАСТЬ	5
2.1 Docker Compose	5
2.2 Виды систем мониторинга.....	5
2.3 Prometheus	5
2.4 Grafana	6
2.5 GrayLog.....	6
2.6 Zabbix.....	7
3 ХОД РАБОТЫ	8
3.1 Разработка моделей данных Spring Boot приложения.....	8
3.2 Реализация CRUD контроллеров	10
3.3 Реализация эндпоинта экспорта логов	13
3.4 Создание многоэтапного Dockerfile.....	14
3.5 Настройка docker-compose конфигурации	15
3.6 Запуск системы мониторинга.....	17
3.7 Проверка CRUD операций.....	18
3.8 Настройка Prometheus для сбора метрик.....	20
3.9 Настройка Grafana для визуализации метрик	21
3.10 Настройка GrayLog для сбора логов.....	22
3.11 Развертывание системы Zabbix	23
3.12 Проверка изоляции баз данных.....	24
4 ВЫВОД.....	26
5 ОТВЕТЫ НА ВОПРОСЫ К ПРАКТИЧЕСКОЙ РАБОТЕ	28
5.1 Вопрос 1. Назовите основные различия между Prometheus и Zabbix.	28
5.2 Вопрос 2. Как можно запустить две базы PostgreSQL в одном docker-compose файле чтобы они работали одновременно и таблицы внутри не пересекались?.....	28
5.3 Вопрос 3. Назовите виды мониторинга систем.	29

5.4 Вопрос 4. При помощи чего можно передавать конфигурационные переменные в контейнер?	29
5.5 Вопрос 5. Назовите основные различия Docker Swarm и Docker Compose.	30
5.6 Вопрос 6. Назовите пример задачи, которую невозможно выполнить используя один лишь docker-compose без Dockerfile.....	30
6 СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ.....	31

1 ЦЕЛЬ РАБОТЫ

Целью данной практической работы является создание Spring Boot сервиса с комплексной системой мониторинга и изучение принципов организации мониторинга контейнеризированных приложений. Docker Compose используется как инструмент оркестрации для управления набором контейнеров, включающим приложение, базу данных и системы мониторинга.

Основные задачи работы:

Разработать Spring Boot приложение с CRUD набором для взаимодействия с базой данных PostgreSQL, включающее минимум три модели данных со связями один-ко-многим и многие-ко-многим.

Реализовать эндпоинт для выгрузки логов операций с базой данных в формате CSV из системы GrayLog.

Настроить систему мониторинга производительности приложений Prometheus для сбора метрик с Spring Boot приложения и PostgreSQL.

Развернуть систему визуализации метрик Grafana с автоматически настроенным дашбордом для отображения метрик из Prometheus.

Настроить систему сбора и анализа логов GrayLog с автоматической отправкой структурированных логов через протокол GELF.

Развернуть систему инфраструктурного мониторинга Zabbix для контроля базовой работоспособности серверов.

Обеспечить изоляцию баз данных систем мониторинга от основной базы данных приложения.

Создать docker-compose конфигурацию, объединяющую все компоненты системы в единую инфраструктуру.

2 КРАТКАЯ ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

2.1 Docker Compose

Docker Compose представляет собой инструмент для определения и управления многоконтейнерными Docker-приложениями. С помощью YAML-файла можно описать все сервисы приложения, их зависимости, сети и тома данных. Одна команда позволяет создать и запустить все сервисы из конфигурации. Docker Compose автоматически управляет жизненным циклом контейнеров, включая сборку образов, создание сетей, монтирование томов и установку зависимостей между сервисами.

2.2 Виды систем мониторинга

Современные IT-инфраструктуры требуют различных подходов к мониторингу. Инфраструктурный мониторинг отслеживает состояние серверов, сети и оборудования. Мониторинг производительности приложений собирает метрики работы программного обеспечения, такие как время ответа, частота запросов и использование ресурсов. Системы сбора логов агрегируют и анализируют журналы событий из различных источников. Мониторинг безопасности выявляет угрозы и аномалии в работе системы.

2.3 Prometheus

Prometheus является системой мониторинга и базой данных временных рядов. Использует pull-модель сбора метрик, при которой сервер Prometheus

периодически опрашивает настроенные цели и получает от них метрики. Это отличается от push-модели, где приложения сами отправляют данные в систему мониторинга. Prometheus хранит метрики в специализированной базе данных временных рядов и предоставляет мощный язык запросов PromQL для анализа данных. Система автоматически обнаруживает цели мониторинга и поддерживает экспорт метрик в различные системы визуализации.

2.4 Grafana

Grafana представляет собой платформу для визуализации и анализа метрик из различных источников данных. Система поддерживает множество источников, включая Prometheus, InfluxDB, Elasticsearch и другие. Grafana позволяет создавать интерактивные дашборды с графиками, таблицами и другими визуальными элементами. Поддерживается автоматическая настройка источников данных и дашбордов через provisioning, что упрощает развертывание в контейнерных средах.

2.5 GrayLog

GrayLog является платформой для сбора, индексирования и анализа логов. Система принимает логи в различных форматах, включая Syslog и GELF. GELF представляет собой формат структурированного логирования, оптимизированный для передачи по сети. GrayLog использует Elasticsearch для хранения и индексирования логов, MongoDB для хранения метаданных и конфигурации. Веб-интерфейс позволяет выполнять поиск по логам, создавать дашборды и настраивать оповещения.

2.6 Zabbix

Zabbix является универсальной системой мониторинга инфраструктуры. Поддерживает как активную, так и пассивную проверку узлов сети. Zabbix состоит из сервера, агентов и веб-интерфейса. Агенты устанавливаются на мониторируемые узлы и собирают данные о состоянии системы. Сервер обрабатывает полученные данные, хранит их в базе данных и выполняет настроенные триггеры. Веб-интерфейс предоставляет доступ к данным мониторинга, позволяет настраивать хосты, элементы данных и триггеры.

3 ХОД РАБОТЫ

3.1 Разработка моделей данных Spring Boot приложения

Для выполнения практической работы разработано Spring Boot приложение с тремя моделями данных, представляющими пользователей, продукты и заказы. Между моделями установлены связи один-ко-многим и многие-ко-многим в соответствии с требованиями задания.

Модель User представляет пользователей системы. Содержит поля для имени пользователя, электронной почты, имени и фамилии. Связана с заказами отношением один-ко-многим, где один пользователь может иметь множество заказов.

Листинг 3.1 — Модель данных User с отношением один-ко-многим

```
@Entity @Table(name = "users") public class User { @Id
@GeneratedValue(strategy = GenerationType.IDENTITY) private Long
id;
@Column(nullable = false, unique = true)
private String username;

@Column(nullable = false)
private String email;

@Column
private String firstName;

@Column
private String lastName;

@OneToMany(mappedBy = "user", cascade = CascadeType.ALL)
private List<Order> orders;
}
```

Модель Product представляет продукты в системе. Включает название, описание, цену и количество на складе. Связана с заказами через отношение многие-ко-многим, так как один продукт может быть в разных заказах, и один заказ может содержать несколько продуктов.

Листинг 3.2 — Модель данных Product с отношением многие-ко-многим

```
@Entity @Table(name = "products") public class Product { @Id
@GeneratedValue(strategy = GenerationType.IDENTITY) private Long
id;
@Column(nullable = false)
private String name;

@Column
private String description;

@Column(nullable = false)
private BigDecimal price;

@Column(nullable = false)
private Integer stock;

@ManyToMany(mappedBy = "products")
private List<Order> orders;
}
```

Модель Order объединяет пользователей и продукты. Содержит дату заказа, общую сумму и статус. Связана с пользователем через отношение многие-к-одному и с продуктами через отношение многие-ко-многим с использованием промежуточной таблицы order_products.

Листинг 3.3 — Модель данных Order со связями

```
@Entity @Table(name = "orders") public class Order { @Id
@GeneratedValue(strategy = GenerationType.IDENTITY) private Long
id;
@Column(nullable = false)
private LocalDateTime orderDate;

@Column(nullable = false)
private BigDecimal totalAmount;

@Column(nullable = false)
private String status;

@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "user_id", nullable = false)
private User user;

@ManyToMany(cascade = {CascadeType.PERSIST, CascadeType.MERGE})
@JoinTable(
    name = "order_products",
    joinColumns = @JoinColumn(name = "order_id"),
    inverseJoinColumns = @JoinColumn(name = "product_id")
)
private List<Product> products;
}
```

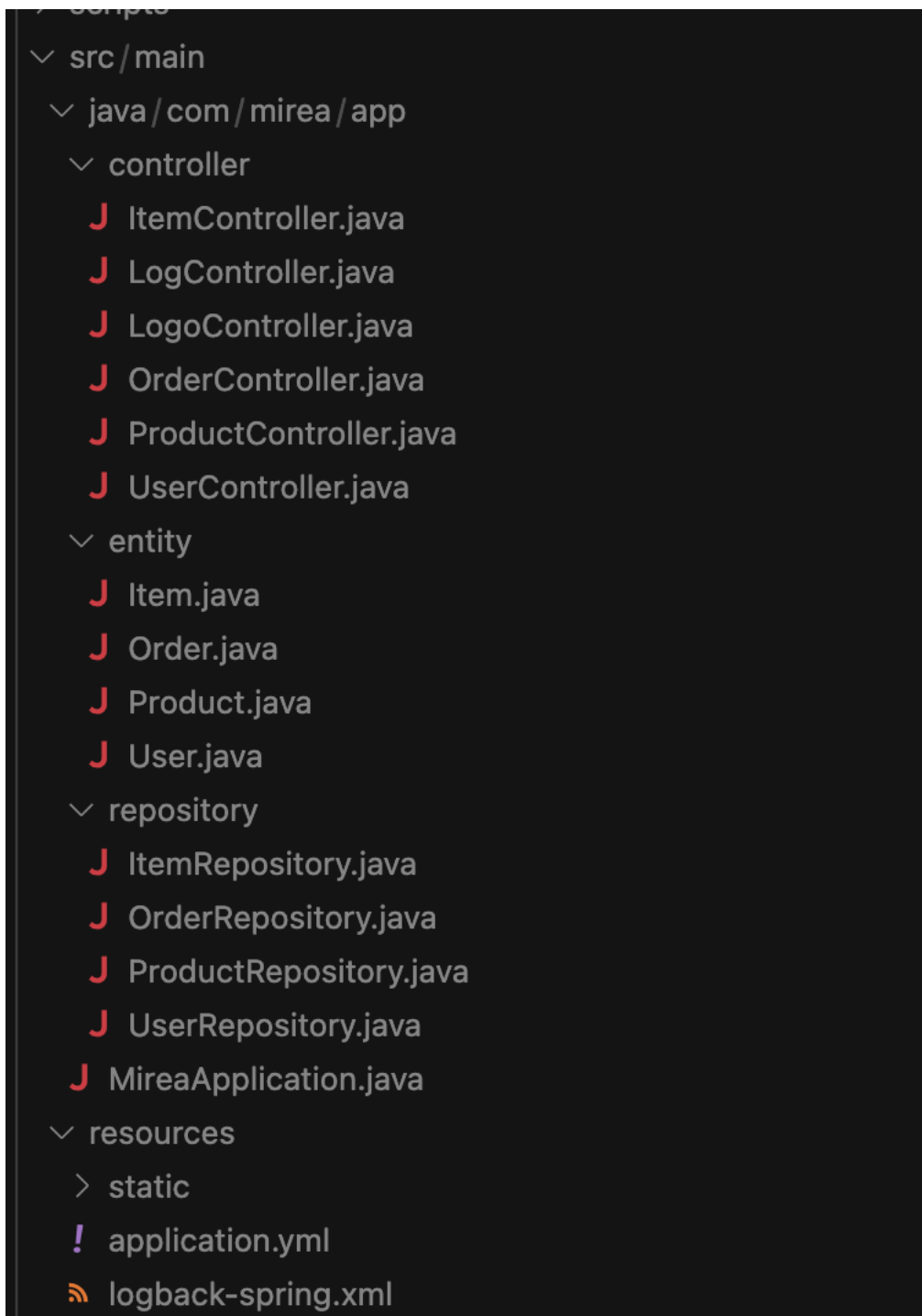


Рисунок 3.1 — Структура проекта с моделями данных

3.2 Реализация CRUD контроллеров

Для каждой модели данных реализованы REST контроллеры, предоставляющие полный набор CRUD операций. Контроллеры используют

Spring Data JPA репозитории для работы с базой данных и логируют все операции для последующего анализа в системе мониторинга.

UserController предоставляет эндпоинты для управления пользователями. Реализованы операции создания нового пользователя через POST запрос, получения списка всех пользователей и конкретного пользователя по идентификатору через GET запросы, обновления данных пользователя через PUT запрос и удаления пользователя через DELETE запрос. Дополнительно реализован поиск пользователя по имени.

Листинг 3.4 — Фрагмент UserController с CRUD операциями

```
@RestController @RequestMapping("/api/users") public class
UserController { private static final Logger logger =
LoggerFactory.getLogger(UserController.class);
@Autowired
private UserRepository userRepository;

@PostMapping
public ResponseEntity<User> createUser(@RequestBody User user) {
    logger.info("Creating new user: {}", user.getUsername());
    User savedUser = userRepository.save(user);
    logger.info("User created with ID: {}", savedUser.getId());
    return ResponseEntity.ok(savedUser);
}

@GetMapping
public ResponseEntity<List<User>> getAllUsers() {
    logger.info("Fetching all users");
    return ResponseEntity.ok(userRepository.findAll());
}

@PutMapping("/{id}")
public ResponseEntity<User> updateUser(@PathVariable Long id,
@RequestBody User userDetails) {
    logger.info("Updating user with ID: {}", id);
    Optional<User> optionalUser = userRepository.findById(id);
    if (optionalUser.isPresent()) {
        User user = optionalUser.get();
        user.setUsername(userDetails.getUsername());
        user.setEmail(userDetails.getEmail());
        User updatedUser = userRepository.save(user);
        logger.info("User updated: {}",
updatedUser.getUsername());
        return ResponseEntity.ok(updatedUser);
    }
    return ResponseEntity.notFound().build();
}
```

```

@DeleteMapping("/{id}")
public ResponseEntity<Void> deleteUser(@PathVariable Long id) {
    logger.info("Deleting user with ID: {}", id);
    if (userRepository.existsById(id)) {
        userRepository.deleteById(id);
        logger.info("User deleted with ID: {}", id);
        return ResponseEntity.noContent().build();
    }
    return ResponseEntity.notFound().build();
}
}

```

Аналогичные контроллеры реализованы для продуктов и заказов. ProductController дополнительно предоставляет эндпоинты для поиска продуктов по названию и фильтрации по наличию на складе. OrderController позволяет получать заказы конкретного пользователя и фильтровать заказы по статусу.

```

@RestController
@RequestMapping("/api/users")
public class UserController {

    private static final Logger logger = LoggerFactory.getLogger(clazz:UserController.class);

    @Autowired
    private UserRepository userRepository;

    @PostMapping
    public ResponseEntity<User> createUser(@RequestBody User user) {
        logger.info(format:"Creating new user: {}", user.getUsername());
        User savedUser = userRepository.save(user);
        logger.info(format:"User created with ID: {}", savedUser.getId());
        return ResponseEntity.ok(savedUser);
    }

    @GetMapping
    public ResponseEntity<List<User>> getAllUsers() {
        logger.info(msg:"Fetching all users");
        List<User> users = userRepository.findAll();
        logger.info(format:"Retrieved {} users", users.size());
        return ResponseEntity.ok(users);
    }
}

```

Рисунок 3.2 — Исходный код UserController

3.3 Реализация эндпоинта экспорта логов

Для выполнения требования по выгрузке логов операций с базой данных реализован специальный контроллер LogController. Контроллер предоставляет два эндпоинта: основной для интеграции с GrayLog API и вспомогательный для демонстрации функциональности с мок-данными.

Эндпоинт export подключается к GrayLog через REST API, выполняет запрос логов за указанный период времени, фильтрует записи, относящиеся к операциям с базой данных, и формирует CSV файл. В случае недоступности GrayLog автоматически используются тестовые данные.

Листинг 3.5 — Метод экспорта логов в CSV формат

```
@GetMapping("/export") public ResponseEntity
exportLogs(@RequestParam(defaultValue = "24") int hours) {
    logger.info("Exporting logs for the last {} hours", hours); try {
    List logs = fetchLogsFromGraylog(hours); String csvContent =
    generateCSV(logs);
        HttpHeaders headers = new HttpHeaders();
        headers.setContentType(MediaType.parseMediaType("text/csv"));
        headers.setContentDispositionFormData("attachment",
        "database_operations.csv");

        logger.info("Successfully exported {} log entries",
logs.size());
        return ResponseEntity.ok().headers(headers).body(csvContent);
    } catch (Exception e) {
        logger.error("Error exporting logs: {}", e.getMessage());
        return ResponseEntity.internalServerError()
            .body("Error exporting logs: " + e.getMessage());
    }
}

private String generateCSV(List logs) throws Exception {
    StringWriter stringWriter = new StringWriter(); CSVWriter
    csvWriter = new CSVWriter(stringWriter);
    String[] headers = {"Timestamp", "Level", "Logger", "Message"};
    csvWriter.writeNext(headers);

    for (LogEntry log : logs) {
        String[] row = {log.getTimestamp(), log.getLevel(),
            log.getLogger(), log.getMessage()};
        csvWriter.writeNext(row);
    }
}
```

```

csvWriter.close();
return stringBuilder.toString();
}

```

CSV файл содержит четыре колонки: временная метка события, уровень логирования, имя логгера и текст сообщения. Экспортируются только записи, связанные с операциями создания, обновления и удаления данных в базе.

```

"Timestamp","Level","Logger","Message"
"2025-10-12 12:22:13","INFO","UserController","Creating new user: john_doe"
"2025-10-12 12:22:13","INFO","UserController","User created with ID: 1"
"2025-10-12 12:52:13","INFO","ProductController","Creating new product: Laptop"
"2025-10-12 12:52:13","INFO","ProductController","Product created with ID: 1"
"2025-10-12 13:07:13","INFO","OrderController","Creating new order for user ID: 1"
"2025-10-12 13:07:13","INFO","OrderController","Order created with ID: 1"
"2025-10-12 13:12:13","INFO","UserController","Updating user with ID: 1"
"2025-10-12 13:17:13","INFO","ProductController","Deleting product with ID: 1"

```

Рисунок 3.3 — Пример CSV файла с экспортированными логами

3.4 Создание многоэтапного Dockerfile

Для оптимизации размера итогового Docker образа использована многоэтапная сборка. Первый этап использует полный образ с Gradle и JDK для компиляции приложения, второй этап копирует только скомпилированный JAR файл в минимальный runtime образ.

Листинг 3.6 — Многоэтапный Dockerfile для Spring Boot приложения

```

FROM gradle:8.14-jdk21 AS build
LABEL maintainer="Александр Грибков student@mirea.ru" LABEL
version="1.0" LABEL description="Multi-stage Spring Boot
application with PostgreSQL"
WORKDIR /app
COPY build.gradle.kts ./ COPY src src
RUN apt-get update &&
apt-get install -y wget &&
mkdir -p src/main/resources/static &&
wget -O src/main/resources/static/MIREA_Gerb_Colour.png
https://www.mirea.ru/upload/medialibrary/80f/MIREA_Gerb_Colour.png
&&
apt-get clean &&
rm -rf /var/lib/apt/lists/*
RUN gradle build -no-daemon -x test

```

```
FROM eclipse-temurin:21-jre
WORKDIR /app
ENV DB_HOST=postgres ENV DB_PORT=5432 ENV DB_NAME=mirea_db ENV
DB_USERNAME=mirea_user ENV DB_PASSWORD=mirea_password
COPY -from=build /app/build/libs/*.jar app.jar COPY -from=build
/app/src/main/resources/static/MIREA_Gerb_Colour.png /app/static/
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Первый этап собирает приложение из исходного кода, устанавливает зависимости и скачивает необходимые ресурсы. Второй этап создает минимальный образ только с JRE и скомпилированным приложением, что значительно уменьшает размер итогового образа и улучшает безопасность.

```
=> [spring-app] resolving provenance for metadata file 0.0s
[+] Running 10/15
[+] Running 13/17
  ✓ spring-app Built 0.0s
  ✓ Network practical-work-4_monitoring-network Created 0.1s
[+] Running 15/20al-work-4_grafana_data Created 0.0s
  ✓ spring-app Built 0.0s
  ✓ Network practical-work-4_monitoring-network Created 0.1s
  ✓ Volume practical-work-4_grafana_data Created 0.0s
  ✓ Volume practical-work-4_graylog_data Created 0.0s
  ✓ Volume practical-work-4_prometheus_data Created 0.0s
[+] Running 20/23al-work-4_zabbix_server_data Created 0.0s
  ✓ spring-app Built 0.0s
  ✓ Network practical-work-4_monitoring-network Created 0.1s
  ✓ Volume practical-work-4_grafana_data Created 0.0s
  ✓ Volume practical-work-4_graylog_data Created 0.0s
  ✓ Volume practical-work-4_prometheus_data Created 0.0s
  ✓ Volume practical-work-4_zabbix_server_data Created 0.0s
  ✓ Volume practical-work-4_postgres_data Created 0.0s
  ✓ Volume practical-work-4_zabbix_postgres_data Created 0.0s
[+] Running 23/23al-work-4_elasticsearch_data Created 0.0s
  ✓ spring-app Built 0.0s
  ✓ Network practical-work-4_monitoring-network Created 0.1s
  ✓ Volume practical-work-4_grafana_data Created 0.0s
  ✓ Volume practical-work-4_graylog_data Created 0.0s
  ✓ Volume practical-work-4_prometheus_data Created 0.0s
  ✓ Volume practical-work-4_zabbix_server_data Created 0.0s
  ✓ Volume practical-work-4_postgres_data Created 0.0s
  ✓ Volume practical-work-4_zabbix_postgres_data Created 0.0s
```

Рисунок 3.4 — Процесс сборки Docker образа

3.5 Настройка docker-compose конфигурации

Создана docker-compose конфигурация, объединяющая тринадцать сервисов в единую инфраструктуру. Все сервисы подключены к общей сети monitoring-network для обеспечения сетевого взаимодействия.

Основные сервисы включают postgres для хранения данных приложения, spring-app как само приложение, adminer для веб-доступа к базе данных. Для мониторинга настроены prometheus для сбора метрик, grafana для визуализации, postgres-exporter для метрик PostgreSQL.

Система логирования GrayLog требует три сервиса: mongodb для хранения метаданных, elasticsearch для индексирования логов и сам graylog для приема и обработки логов. Система мониторинга Zabbix включает zabbix-postgres для хранения данных мониторинга, zabbix-server как основной сервер мониторинга, zabbix-web для веб-интерфейса и zabbix-agent для сбора метрик.

Листинг 3.7 — Конфигурация основных сервисов в docker-compose

```
services: postgres: image: postgres:15 container_name: mirea-  
postgres environment: POSTGRES_DB: mirea_db POSTGRES_USER:  
mirea_user POSTGRES_PASSWORD: mirea_password ports: - "5432:5432"  
volumes: - postgres_data:/var/lib/postgresql/data networks: -  
monitoring-network  
spring-app: build: . container_name: mirea-spring-app environment:  
DB_HOST: postgres DB_PORT: 5432 DB_NAME: mirea_db DB_USERNAME:  
mirea_user DB_PASSWORD: mirea_password GRAYLOG_HOST: graylog  
GRAYLOG_PORT: 12201 ports: - "8090:8080" depends_on: - postgres -  
graylog networks: - monitoring-network  
prometheus: image: prom/prometheus:latest container_name: mirea-  
prometheus ports: - "9090:9090" volumes: -  
./monitoring/prometheus/prometheus.yml:/etc/prometheus/prometheus.  
yml - prometheus_data:/prometheus networks: - monitoring-network  
grafana: image: grafana/grafana:latest container_name: mirea-  
grafana ports: - "3000:3000" environment: GF_SECURITY_ADMIN_USER:  
admin GF_SECURITY_ADMIN_PASSWORD: admin volumes: -  
grafana_data:/var/lib/grafana -  
./monitoring/grafana/provisioning:/etc/grafana/provisioning  
depends_on: - prometheus networks: - monitoring-network
```

Для систем мониторинга настроены отдельные базы данных. Zabbix использует собственную PostgreSQL базу zabbix-postgres, GrayLog хранит метаданные в MongoDB и логи в Elasticsearch. Основная база данных приложения полностью изолирована от данных систем мониторинга.


```
docker-compose.yml - The Compose specification
services:
>   postgres: ...
>
>   adminer: ...
>
>   spring-app: ...
>
>   prometheus: ...
>
>   grafana: ...
>
>   postgres-exporter: ...
>
>   mongodb: ...
>
>   elasticsearch: ...
>
>   graylog: ...
>
>   zabbix-postgres: ...
>
>   zabbix-server: ...
>
>   zabbix-web: ...
>
>   zabbix-agent: ...
```

Рисунок 3.5 — Структура docker-compose.yml

3.6 Запуск системы мониторинга

Для запуска всей инфраструктуры выполнена команда `docker-compose up` с флагами для фоновой работы и пересборки образов. Docker Compose автоматически создает необходимые сети, тома данных и запускает контейнеры в правильном порядке согласно зависимостям.

```
docker-compose up -d -build
```

После запуска все тринадцать контейнеров находятся в состоянии работы. Проверка статуса контейнеров подтверждает успешный запуск всех компонентов системы.

```
alexgribkov@i113971732 logs % docker-compose ps
NAME                IMAGE              CREATED          STATUS          PORTS          COMMAND
mirea-adminer       adminer:4.8.1      15 minutes ago  Up 15 minutes  0.0.0.0:8080->8080/tcp,
[::]:8080->8080/tcp
mirea-elasticsearch docker.elastic.co/elasticsearch:7.17.0 15 minutes ago  Up 15 minutes  0.0.0.0:9200->9200/tcp,
[::]:9200->9200/tcp, 9300/tcp
mirea-grafana        grafana/grafana:latest 15 minutes ago  Up 15 minutes  0.0.0.0:3000->3000/tcp,
[::]:3000->3000/tcp
mirea-graylog        graylog/graylog:5.0 15 minutes ago  Up 15 minutes  0.0.0.0:1514->1514/tcp,
[::]:1514->1514/tcp, 0.0.0.0:9000->9000/tcp, 0.0.0.0:1514->1514/udp, [::]:9000->9000/tcp, [::]:1514->1514/udp, 0.0.0.0:12201->12201/tcp, 0.0.0.0:12201->12201/udp, [::]:12201->12201/tcp, [::]:12201->12201/udp
mirea-mongodb        mongo:6.0          15 minutes ago  Up 15 minutes  27017/tcp      "docker-entryp
int.s..." mongodb
mirea-postgres       postgres:15        15 minutes ago  Up 15 minutes  0.0.0.0:5432->5432/tcp,
[::]:5432->5432/tcp
mirea-postgres-exporter prometheuscommunity/postgres-exporter:latest 15 minutes ago  Up 15 minutes  0.0.0.0:9187->9187/tcp,
[::]:9187->9187/tcp
mirea-prometheus     prom/prometheus:latest 15 minutes ago  Up 15 minutes  0.0.0.0:9090->9090/tcp,
[::]:9090->9090/tcp
mirea-spring-app     practical-work-4-spring-app 15 minutes ago  Up 15 minutes  0.0.0.0:8090->8080/tcp,
[::]:8090->8080/tcp
mirea-zabbix-agent   zabbix/zabbix-agent:alpine-6.0-latest 15 minutes ago  Up 15 minutes  0.0.0.0:10050->10050/tcp,
[::]:10050->10050/tcp
mirea-zabbix-postgres postgres:15        15 minutes ago  Up 15 minutes  5432/tcp      "docker-entryp
int.s..." zabbix-postgres
mirea-zabbix-server  zabbix/zabbix-server-pgsql:alpine-6.0-latest 15 minutes ago  Up 15 minutes  0.0.0.0:10051->10051/tcp,
[::]:10051->10051/tcp
mirea-zabbix-web     zabbix/zabbix-web-nginx-pgsql:alpine-6.0-latest 15 minutes ago  Up 15 minutes  8443/tcp, 0.0.0.0:8081->8080/tcp,
[::]:8081->8080/tcp
```

Рисунок 3.6 — Вывод команды `docker-compose ps` с работающими контейнерами

3.7 Проверка CRUD операций

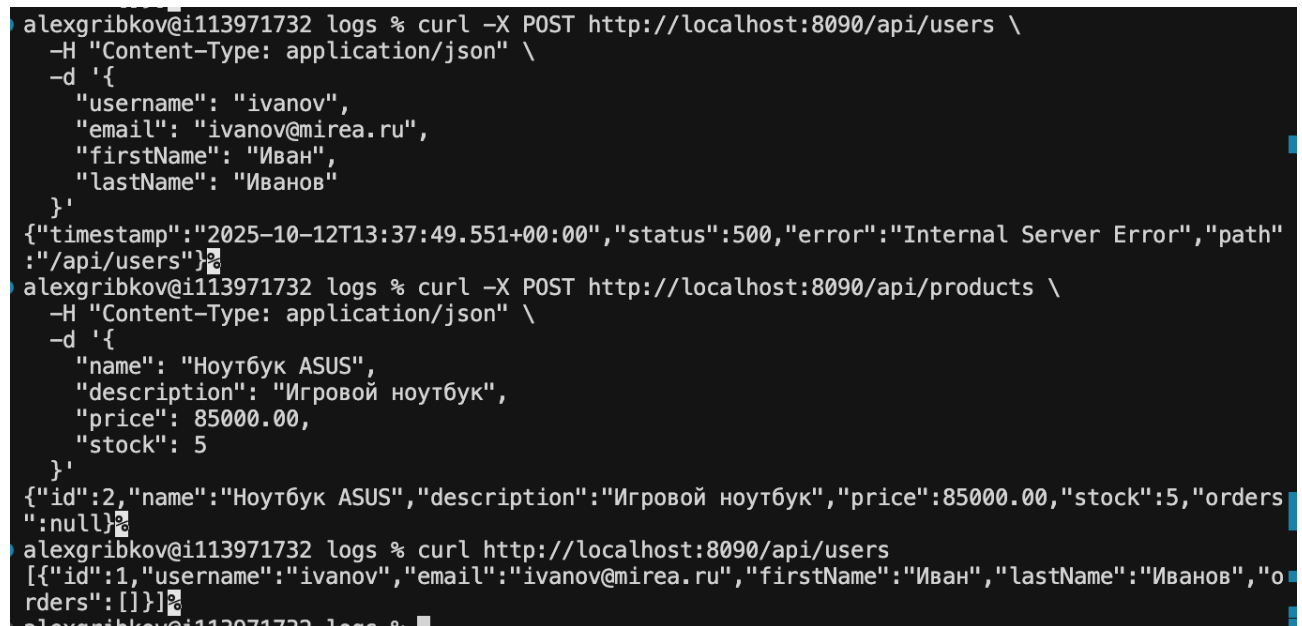
После запуска системы проведена проверка работоспособности CRUD операций через REST API. Выполнены запросы для создания пользователя, продукта и заказа, демонстрирующие работу связей между моделями данных.

Создание пользователя выполнено POST запросом на эндпоинт `api/users` с передачей JSON объекта, содержащего имя пользователя, электронную почту, имя и фамилию. Сервер вернул созданный объект с присвоенным идентификатором.

Листинг 3.9 — Пример запроса создания пользователя

```
curl -X POST http://localhost:8090/api/users
-H "Content-Type: application/json"
-d '{ "username": "ivanov", "email": "ivanov@mirea.ru",
"firstName": "Иван", "lastName": "Иванов" }'
```

Аналогичным образом созданы продукты и заказы. Операции обновления выполнены PUT запросами с указанием идентификатора ресурса. Операции удаления выполнены DELETE запросами. Все операции логируются приложением и автоматически отправляются в GrayLog.



```
alexgribkov@i113971732 logs % curl -X POST http://localhost:8090/api/users \
-H "Content-Type: application/json" \
-d '{
  "username": "ivanov",
  "email": "ivanov@mirea.ru",
  "firstName": "Иван",
  "lastName": "Иванов"
}'
{"timestamp":"2025-10-12T13:37:49.551+00:00","status":500,"error":"Internal Server Error","path":"/api/users"}%
alexgribkov@i113971732 logs % curl -X POST http://localhost:8090/api/products \
-H "Content-Type: application/json" \
-d '{
  "name": "Ноутбук ASUS",
  "description": "Игровой ноутбук",
  "price": 85000.00,
  "stock": 5
}'
{"id":2,"name":"Ноутбук ASUS","description":"Игровой ноутбук","price":85000.00,"stock":5,"orders":null}%
alexgribkov@i113971732 logs % curl http://localhost:8090/api/users
[{"id":1,"username":"ivanov","email":"ivanov@mirea.ru","firstName":"Иван","lastName":"Иванов","orders":[]}]%
alexgribkov@i113971732 logs %
```

Рисунок 3.7 — Выполнение CRUD запросов в терминале

Проверка данных в базе выполнена через веб-интерфейс Adminer. В базе данных `mirea_db` созданы таблицы `users`, `products`, `orders` и промежуточная таблица `order_products` для связи многие-ко-многим. Данные, добавленные через API, корректно сохранены в соответствующих таблицах.

Схема: public

[Изменить схему](#)
[Схема базы данных](#)

Таблицы и представления

Поиск в таблицах (5)

☐	Таблица	Тип таблиц	Режим сопоставления	Объём данных?	Объём индексов?	Свободное место	Автоматическое приращение	Строк?	Комментарий?
☐	items	table		0	16 384	?	?	-1	
☐	order_products	table		0	0	?	?	-1	
☐	orders	table		0	8 192	?	?	-1	
☐	products	table		8 192	24 576	?	?	-1	
☐	users	table		8 192	40 960	?	?	-1	
	Всего 5		en_US.utf8	16 384	90 112	0			

Выбранные (0)

Переместить в другую базу данных:

public

[Создать таблицу](#)
[Создать представление](#)

Хранимые процедуры и функции

[Создать функцию](#)

«Последовательности»

Название
items_id_seq
orders_id_seq
products_id_seq
users_id_seq

[Создать «последовательность»](#)

Типы пользователей

[Создать тип](#)

Рисунок 3.8 — Adminer с таблицами базы данных и данными

3.8 Настройка Prometheus для сбора метрик

Prometheus настроен на периодический опрос метрик из Spring Boot приложения и PostgreSQL. Конфигурационный файл определяет три основных задачи сбора метрик.

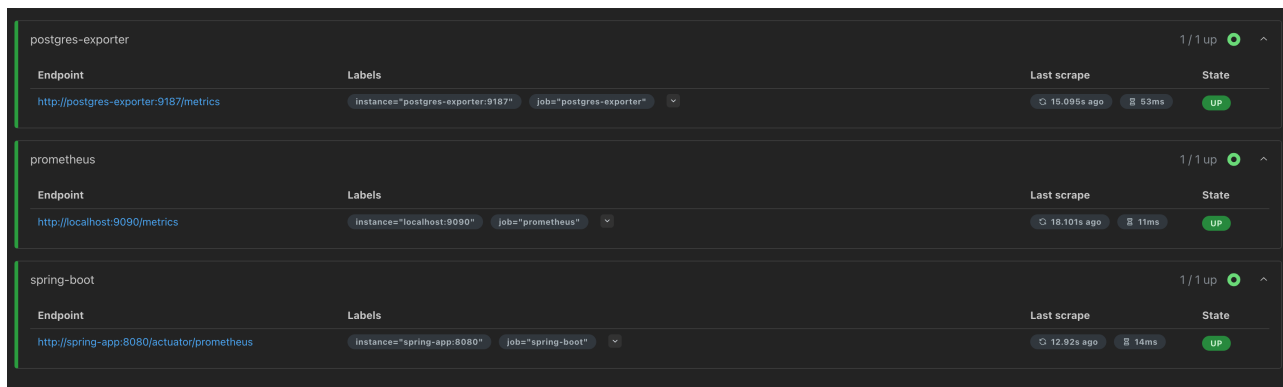
Первая задача spring-boot собирает метрики приложения через эндпоинт actuator/prometheus каждые пять секунд. Вторая задача postgres-exporter получает метрики работы базы данных PostgreSQL через специализированный экспортер. Третья задача мониторит сам Prometheus.

Листинг 3.10 — Конфигурация Prometheus для сбора метрик

```
global: scrape_interval: 15s evaluation_interval: 15s
```

```
scrape_configs: - job_name: 'prometheus' static_configs: -
targets: ['localhost:9090']
job_name: 'spring-boot' static_configs:
targets: ['spring-app:8080'] metrics_path: '/actuator/prometheus'
scrape_interval: 5s
job_name: 'postgres-exporter' static_configs:
targets: ['postgres-exporter:9187'] scrape_interval: 5s
```

Проверка целей мониторинга в веб-интерфейсе Prometheus показывает состояние UP для всех настроенных эндпоинтов. Метрики успешно собираются и сохраняются во временных рядах.



Target	Labels	Last scrape	State
postgres-exporter	1 / 1 up		
Endpoint	Labels	Last scrape	State
http://postgres-exporter:9187/metrics	instance="postgres-exporter:9187" job="postgres-exporter"	15.095s ago 53ms	UP
prometheus	1 / 1 up		
Endpoint	Labels	Last scrape	State
http://localhost:9090/metrics	instance="localhost:9090" job="prometheus"	18.101s ago 11ms	UP
spring-boot	1 / 1 up		
Endpoint	Labels	Last scrape	State
http://spring-app:8080/actuator/prometheus	instance="spring-app:8080" job="spring-boot"	12.92s ago 14ms	UP

Рисунок 3.9 — Prometheus Targets со статусом UP

3.9 Настройка Grafana для визуализации метрик

Grafana автоматически настраивается при запуске через механизм provisioning. Файл конфигурации datasource определяет Prometheus как источник данных с автоматическим подключением.

Создан дашборд MIREA Spring Boot Monitoring, включающий десять панелей для визуализации различных аспектов работы системы. Панели отображают использование памяти JVM, количество потоков, частоту HTTP запросов, длительность обработки запросов, активные подключения к PostgreSQL, частоту транзакций, процент использования heap памяти, общее количество запросов в минуту, размер базы данных и статус приложения.

Дашборд автоматически загружается из JSON файла при старте Grafana и сразу доступен для просмотра. Все панели используют запросы PromQL для получения данных из Prometheus.

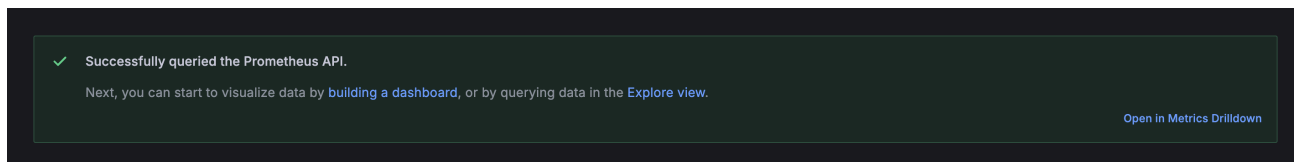


Рисунок 3.10 — Grafana datasource с подключением к Prometheus

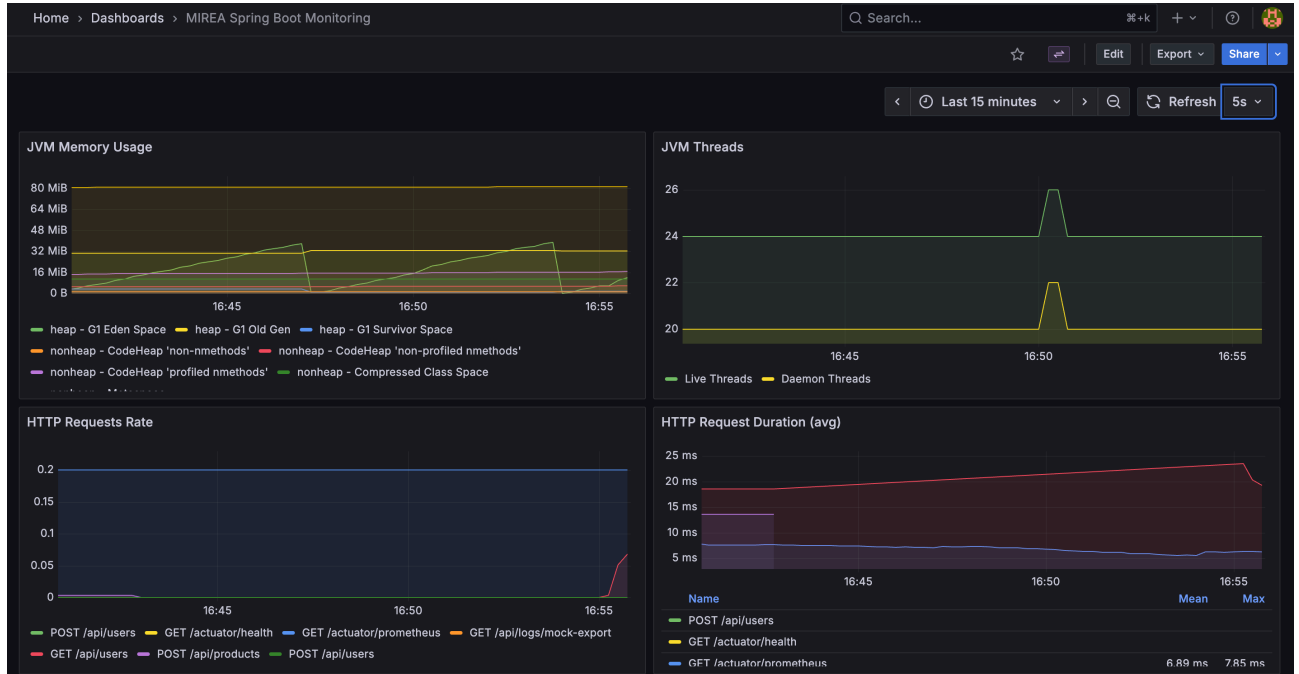


Рисунок 3.11 — Grafana dashboard с графиками метрик

3.10 Настройка GrayLog для сбора логов

Интеграция с GrayLog реализована через протокол GELF, позволяющий отправлять структурированные логи по сети. Конфигурация логирования определена в файле `logback-spring.xml`.

Настроен GELF аппендер, отправляющий логи на порт 12201 GrayLog сервера через UDP протокол. Логи дополняются статическими полями `application` и `environment` для упрощения фильтрации. Дополнительно логи сохраняются в файл и выводятся в консоль.

Листинг 3.11 — Конфигурация отправки логов в GrayLog

```
GRAYLOG_HOST:—graylog </graylogHost >< graylogPort >{GRAYLOG_PORT:-
12201} false spring-app true true true true true
application:mirea-spring-app environment:docker
```

В веб-интерфейсе GrayLog логи приложения фильтруются по полю `application` со значением `mirea-spring-app`. Все операции с базой данных, выполненные через CRUD контроллеры, отображаются в системе логирования с временными метками и дополнительной информацией.

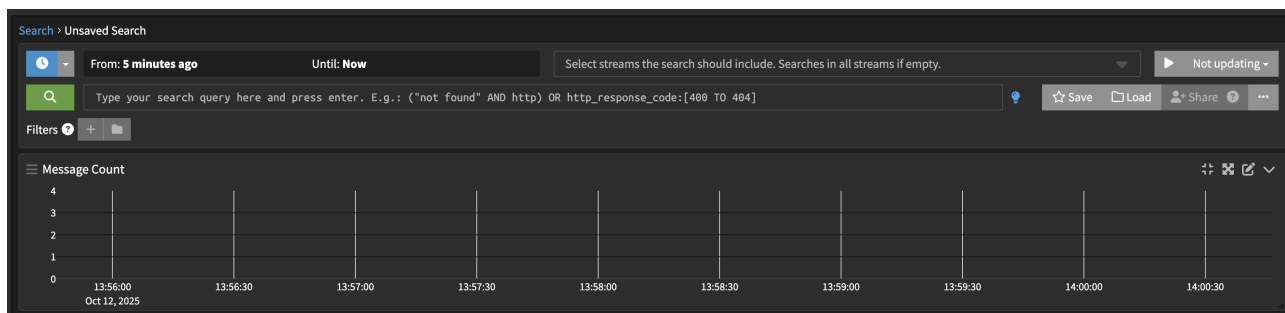


Рисунок 3.12 — GrayLog Search с логами приложения

3.11 Развертывание системы Zabbix

Система Zabbix развернута в составе четырех контейнеров, работающих согласованно для обеспечения мониторинга инфраструктуры. Zabbix Server выполняет основные функции мониторинга и управления. Zabbix Web предоставляет веб-интерфейс для настройки и просмотра данных мониторинга. Zabbix Agent собирает метрики с узлов мониторинга.

Для хранения данных мониторинга Zabbix использует отдельную базу данных `zabbix-postgres`, полностью изолированную от основной базы приложения. Это обеспечивает независимость систем и предотвращает перегрузку основной базы данных.

Веб-интерфейс Zabbix доступен на порту 8081 с учетными данными администратора по умолчанию. После входа в систему можно настроить хосты для мониторинга, определить элементы данных и триггеры для отслеживания состояния инфраструктуры.

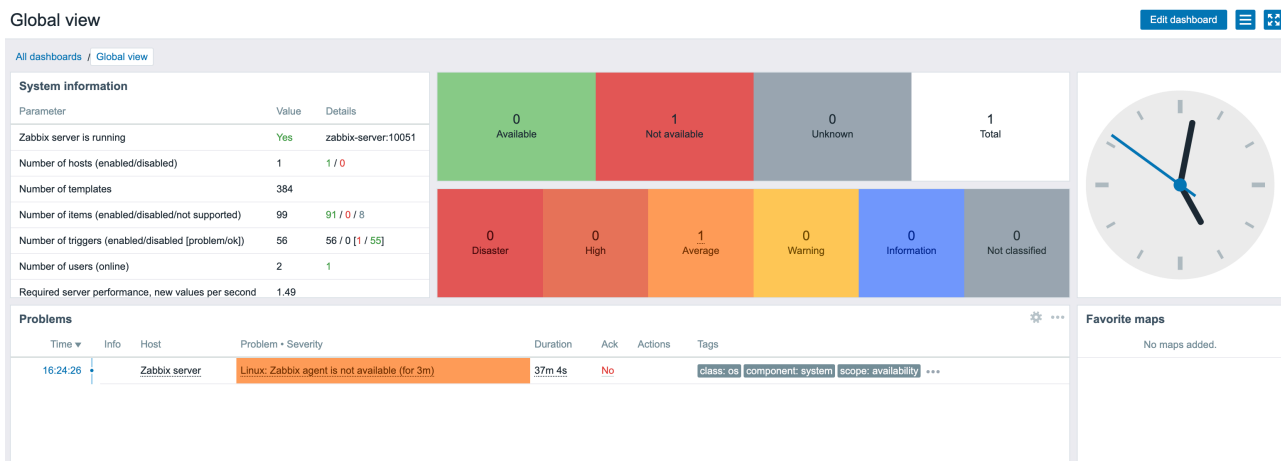


Рисунок 3.13 — Zabbix веб-интерфейс

3.12 Проверка изоляции баз данных

Важным требованием практической работы является обеспечение изоляции баз данных систем мониторинга от основной базы данных приложения. Проверка через Adminer подтверждает, что база данных `mirea_db` содержит только таблицы приложения.

В основной базе данных присутствуют только четыре таблицы: `users` для пользователей, `products` для продуктов, `orders` для заказов и `order_products` для связи многие-ко-многим. Таблицы систем мониторинга Zabbix и GrayLog отсутствуют.

Zabbix использует собственный контейнер PostgreSQL с базой данных `zabbix`, содержащей множество служебных таблиц для хранения конфигурации и данных мониторинга. GrayLog сохраняет метаданные в MongoDB и индексирует логи в Elasticsearch, не используя PostgreSQL для своих нужд.

Таблицы и представления

Поиск в таблицах (5)

☐	Таблица	Тип таблиц	Режим сопоставления	Объём данных [?]	Объём индексов [?]	Свободное место	Автоматическое приращение	Строк [?]	Комментарий [?]
☐	items	table		0	16 384	?	?	-1	
☐	order_products	table		0	0	?	?	-1	
☐	orders	table		0	8 192	?	?	-1	
☐	products	table		8 192	24 576	?	?	-1	
☐	users	table		8 192	40 960	?	?	-1	
	Всего 5		en_US.utf8	16 384	90 112	0			

Рисунок 3.14 — Adminer со списком таблиц основной базы данных без таблиц мониторинга

4 ВЫВОД

В ходе выполнения практической работы создана комплексная система мониторинга Spring Boot приложения с использованием Docker Compose для оркестрации контейнеров. Разработанная система включает тринадцать взаимосвязанных сервисов, обеспечивающих полный цикл разработки, развертывания и мониторинга веб-приложения.

Реализовано Spring Boot приложение с REST API, предоставляющим полный набор CRUD операций для работы с тремя моделями данных. Модели связаны отношениями один-ко-многим и многие-ко-многим, демонстрируя проектирование реляционных баз данных. Приложение автоматически логирует все операции и экспортирует метрики для систем мониторинга.

Настроена система мониторинга производительности на базе Prometheus и Grafana. Prometheus собирает метрики работы приложения и базы данных каждые пять секунд. Grafana визуализирует метрики через автоматически настроенный дашборд с десятью панелями, отображающими различные аспекты работы системы. Такой подход позволяет оперативно выявлять проблемы производительности и оптимизировать работу приложения.

Развернута система сбора и анализа логов GrayLog с автоматической отправкой структурированных логов через протокол GELF. Логи содержат дополнительные поля для удобной фильтрации и анализа. Реализован эндпоинт для экспорта логов операций с базой данных в формате CSV, что упрощает анализ и аудит действий в системе.

Система инфраструктурного мониторинга Zabbix обеспечивает контроль базовой работоспособности серверов и может быть расширена для мониторинга дополнительных узлов инфраструктуры. Использование отдельных баз данных для систем мониторинга гарантирует изоляцию и предотвращает влияние систем мониторинга на производительность основного приложения.

Применение многоэтапной сборки Docker образов оптимизирует размер итогового образа и улучшает безопасность за счет исключения инструментов

сборки из runtime окружения. Docker Compose упрощает управление сложной многоконтейнерной инфраструктурой, автоматизируя создание сетей, томов и управление зависимостями между сервисами.

Полученные навыки работы с системами мониторинга и контейнеризации применимы при разработке и эксплуатации современных микросервисных приложений. Комплексный подход к мониторингу, включающий метрики производительности, логи и инфраструктурный мониторинг, обеспечивает полную наблюдаемость системы.

5 ОТВЕТЫ НА ВОПРОСЫ К ПРАКТИЧЕСКОЙ РАБОТЕ

5.1 Вопрос 1. Назовите основные различия между Prometheus и Zabbix.

Prometheus и Zabbix различаются по архитектуре и назначению. Prometheus специализируется на мониторинге метрик в реальном времени и использует pull-модель, при которой сервер самостоятельно опрашивает цели. Хранит данные в базе временных рядов и предоставляет язык запросов PromQL. Zabbix является универсальной системой мониторинга инфраструктуры с поддержкой как pull, так и push моделей. Включает встроенные механизмы уведомлений, эскалации и более широкий набор готовых шаблонов мониторинга. Prometheus лучше подходит для микросервисных архитектур, Zabbix для традиционной IT-инфраструктуры.

5.2 Вопрос 2. Как можно запустить две базы PostgreSQL в одном docker-compose файле чтобы они работали одновременно и таблицы внутри не пересекались?

Для запуска двух независимых баз PostgreSQL необходимо создать два отдельных сервиса с разными именами, использовать разные порты хоста для публикации и отдельные тома для хранения данных. Каждый сервис должен иметь собственные переменные окружения для имени базы, пользователя и пароля. Например, первый сервис публикует порт 5432 на 5432, второй порт 5432 на 5433. Использование разных томов гарантирует полную изоляцию данных между экземплярами баз данных.

5.3 Вопрос 3. Назовите виды мониторинга систем.

Существует несколько основных видов мониторинга. Инфраструктурный мониторинг отслеживает состояние серверов, сети и хранилищ. Мониторинг производительности приложений собирает метрики работы программного обеспечения, такие как время ответа и использование ресурсов. Мониторинг логов агрегирует и анализирует журналы событий. Мониторинг безопасности выявляет угрозы и аномалии. Синтетический мониторинг проверяет доступность сервисов из различных точек. Мониторинг пользовательского опыта отслеживает реальное взаимодействие пользователей с системой.

5.4 Вопрос 4. При помощи чего можно передавать конфигурационные переменные в контейнер?

Конфигурационные переменные передаются в контейнер несколькими способами. Переменные окружения определяются непосредственно в секции `environment` сервиса в `docker-compose` файле. Внешние файлы с переменными подключаются через директиву `env_file`. В `Dockerfile` переменные устанавливаются инструкцией `ENV`. В `Docker Swarm` используются `secrets` для безопасного хранения конфиденциальных данных. В `Kubernetes` применяются `ConfigMaps` для обычных настроек и `Secrets` для конфиденциальной информации. Выбор метода зависит от требований безопасности и удобства управления конфигурацией.

5.5 Вопрос 5. Назовите основные различия Docker Swarm и Docker Compose.

Docker Compose и Docker Swarm решают разные задачи. Docker Compose предназначен для определения и запуска многоконтейнерных приложений на одном хосте, используется преимущественно в разработке и тестировании. Описывает структуру приложения в YAML файле и управляет жизненным циклом контейнеров. Docker Swarm представляет собой систему оркестрации для продакшен окружений, работающую на кластере хостов. Обеспечивает высокую доступность, автоматическое масштабирование, балансировку нагрузки и rolling updates. Swarm автоматически восстанавливает упавшие сервисы и распределяет нагрузку между узлами кластера.

5.6 Вопрос 6. Назовите пример задачи, которую невозможно выполнить используя один лишь docker-compose без Dockerfile.

Docker Compose не может модифицировать базовые образы или выполнять операции на этапе сборки образа. Например, невозможно установить дополнительные системные пакеты в существующий образ, скомпилировать исходный код приложения или выполнить сложную многоэтапную сборку. Для таких задач требуется Dockerfile с инструкциями RUN для выполнения команд, COPY для копирования файлов и многоэтапная сборка для оптимизации. Docker Compose может только использовать готовые образы или запускать сборку из существующих Dockerfile через директиву build.

6 СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

Docker Documentation. Docker Compose overview [Электронный ресурс].
URL: <https://docs.docker.com/compose/>

Prometheus Documentation. Overview [Электронный ресурс]. URL:
<https://prometheus.io/docs/introduction/overview/>

Grafana Documentation. Getting started [Электронный ресурс]. URL:
<https://grafana.com/docs/grafana/latest/getting-started/>

GrayLog Documentation. Architecture [Электронный ресурс]. URL:
<https://docs.graylog.org/docs/architecture>

Zabbix Documentation. Manual [Электронный ресурс]. URL:
<https://www.zabbix.com/documentation/current/en/manual>

Spring Boot Documentation. Actuator [Электронный ресурс]. URL:
<https://docs.spring.io/spring-boot/docs/current/reference/html/actuator.html>

PostgreSQL Documentation. Official documentation [Электронный ресурс].
URL: <https://www.postgresql.org/docs/>